PYTHON · FROM ZERO · LEARNING LOG

# Python Study Pack

All my lessons, revision guides and drills in one place. **Start with Revision #1**, and cover the answers before checking them.

**8**
guides inside

**08 Nov 2026**
last updated

# 📚 Contents


🧭 **Revision #1: The Journey So Far (Days 01–07 + 23 problems)**
Start here: the 9 revision topics, checklists, what to rewrite

📗 **Python Reference: Types, Ranges & Built-ins**
Types, ranges, built-ins: what each takes and gives back

📕 **The Add · Remove · Loop Book**
Drills: position vs value, add / remove / loop

🗃️ **Data Structures Guide: Lists, Strings, Dictionaries & Parsing Input**
What am I holding? Nested data, parsing, printing

🧯 **How to Solve a Problem Without Panicking**
The STEP method: solve without panicking

🔁 **Big Review #1: Sat 24 + Sun 25 Oct 2026**
Predict-the-output drills: strings, map, dicts

📘 **Day 06: Functions 🧩**
Day 06 lesson: functions

📘 **Day 07: Dictionaries 📖**
Day 07 lesson: dictionaries

# 🧭 Revision #1: The Journey So Far (Days 01–07 + 23 problems)

↑ contents

> **Goal:** before Day 08, make everything from the last 2 weeks feel solid. Then a **long quiz: 10–15 questions per topic**, one topic at a time. Everything here comes from **my** lessons, problems and mistakes.

## 🗺️ The journey in one table

**WEEK**
1 (12–18 Oct)

**LESSONS**
Days 01–06, Day 07 parts 1–2

**PROBLEMS**
9 HackerRank, 2 LeetCode

**BIG MOMENTS**
⭐ ⭐ 2nd star, first LeetCode, STEP-comments method

**WEEK**
2 (19–25 Oct)

**LESSONS**
Day 07 parts 3–4 + Two Sum bonus

**PROBLEMS**
3 HackerRank, 7 LeetCode

**BIG MOMENTS**
first Mediums (2149, Word Order), HackerRank Orchestrate hackathon

**WEEK**
3 (26 Oct–)

**LESSONS**
revision

**PROBLEMS**
LeetCode 1689, Piling Up!

**BIG MOMENTS**
⭐ ⭐ ⭐ ⭐ 4th star, LeetCode Day 10 🔥

---

## 📋 How to revise (per topic)

1. 📖 **Read** the listed sections (read, don't copy). Tick each ✅ line in **"I can..."** only if you could do it with no notes.
2. 🐞 **Reread your own mistakes** for that topic. These are the questions most likely to come up in the quiz 😊
3. ✍️ **Rewrite from memory** the listed problem in a scratch file ( `review.py` ). Stuck for 5 min? Peek at **one line** only.
4. 💬 Say **"ready for topic N"** and you'll get the quiz: 10–15 questions mixing *predict the output*, *find the bug*, *write a few lines* and *explain it like a YouTube viewer*.

**Scoring:** ≥ 80% = ✅ topic done · 60–79% = retry the ❌ ones next session · < 60% = reread and redo that topic. Every quiz is logged in quiz-log.md.

**Suggested order (3–4 sessions, at whatever pace life allows):** - **Session A:** Topics 1–3 (basics, conditions, loops) - **Session B:** Topics 4–5

(strings, lists) ⭐ - **Session C:** Topics 6–8 (functions, dictionaries, parsing and printing) ⭐ - **Session D:** Topic 9 (patterns, errors, problem solving), then **Day 08** 🎉

📗 **Reference for every topic:** python-reference.md: types and their ranges, and for every built-in/method what it takes, what it gives back and when it crashes.

Warm-up drills you already have: review-weekend-1.md (strings, `map`, dicts) and add-remove-loop-book.md (add, remove, loop).

---

## 1️⃣ Basics: print, variables, input, types, maths (Days 01–02)

📖 **Read:** python-reference.md §1, §6 · notes.md: *Day 01*, *Day 02*, *Maths operators* · day01.py, day02.py **I can...** - ☐ explain what a variable is and print several values with commas - ☐ say what type `input()` gives, and convert with `int()`, `float()`, `str()` - ☐ write an f-string with values inside `{}`, and control spaces exactly - ☐ predict `7 / 2`, `7 // 2`, `7 % 2`, `2 ** 3`, and use `% 2 == 0` for "even" - ☐ explain why `"12" + 3` crashes and fix it two ways - ☐ give the range of `int`, `float`, `bool` and say what `0.1 + 0.2` and `-7 // 2` give - ☐ say which values are falsy (and why `bool("False")` is `True`)

🐞 **My mistakes:** missing comma in `print` · f-string spacing `is{age}` · commas in `print` aren't

printed ✍️ **Rewrite:** HackerRank *Arithmetic Operators* and *Python: Division*

## 2️⃣ Conditions: if / elif / else (Day 02)

📖 **Read:** notes.md: *Day 02* · approaches.md: *Python If-Else* **I can...** - ☐ write a flat `if / elif / else` chain (no nesting) and explain why `elif` can **assume** the earlier checks failed - ☐ use `and`, `or`, `not`, `in`, and chained ranges `2 ≤ n ≤ 5` - ☐ pick `>` vs `≥` from the wording ("90+" means `≥ 90`) - ☐ explain why `ch == "a" or "e"` is always True and fix it

🐞 **My mistakes:** `|` used as "or" · `grade == "A"` instead of `"A"` · boundary 90 excluded ✍️ **Rewrite:** HackerRank *Python If-Else* · the Day 06 `grade()` function

## 3️⃣ Loops: for, range, while, break, continue (Day 03)

📖 **Read:** notes.md: *Day 03*, *Loop keywords* · day03.py **I can...** - ☐ list exactly what `range(5)`, `range(1, 6)`, `range(2, 21, 2)`, `range(5, 0, -1)` give - ☐ choose `for` (known count) vs `while` (until something changes) - ☐ make sure every `while` loop changes something so it can end - ☐ explain `break` vs `continue` vs `pass` - ☐ build a total with an ACCUMULATOR (`total = 0`, `total += x`) - ☐ write a loop that goes **backwards** over positions

🐞 **My mistakes:** `range(0, n+1)` off-by-one (HackerRank Loops) · infinite loop (344) · `range(1:n+1)` with a colon (412) · missing `break` (Runner-Up) ✍️ **Rewrite:** HackerRank *Loops* · sum 1 to 100 (three ways: loop, `sum`, Gauss)

## 4️⃣ Strings ⭐ (Day 04)

📖 **Read:** notes.md: *Day 04* · data-structures-guide.md §2, §4, §6 · add-remove-loop-book.md §4 · review-weekend-1.md *Saturday* drills **I can...** - ☐ index and slice: `s[0]`, `s[-1]`, `s[1:4]`, `s[::-1]`, `s[1:]`, `s[:-1]` - ☐ explain why strings can't be changed (`s[0] = "X"` crashes) and rebuild them with `+=` - ☐ use `.upper()` `.lower()` `.strip()` `.replace()` `.split()` `"".join()`, and remember they **return a new string** - ☐ search 3 ways: `x in s` (yes/no), `s.find(x)` (position or -1), `s.count(x)` (how many) - ☐ choose a **letters loop** vs a **positions loop** - ☐ compare strings case-insensitively

🐞 **My mistakes:** `word[ch]` when `ch` is a letter · `.index[i]` with square brackets · `Racecar` failing without `.lower()` · `string.find(ch) == 0` (Merge the Tools) ✍️ **Rewrite:** palindrome · vowel counter · LeetCode 1108 *Defanging an IP* · HackerRank *Merge the Tools!*

## 5️⃣ Lists ⭐ ⭐ (Day 05)

📖 **Read:** notes.md: *Day 05* · add-remove-loop-book.md §0–§2 (**all of it**) · data-structures-guide.md §3 **I can...** - ☐ answer "**position or**

**value?**" for `pop`, `insert`, `remove`, `del`, `in`, `index` - ☐ grow a list with `.append()` (never `ans[0] = ...` on an empty list) - ☐ say what each method **gives back** (`pop` → the item; `append`, `sort`, `remove` → `None`) - ☐ explain `sorted(x)` vs `x.sort()`, and functions (around) vs methods (after the dot) - ☐ loop 4 ways: items, `range(len(...))`, `enumerate`, two pointers - ☐ read nested lists: `students[0][1]` - ☐ explain why removing from a list while looping over it skips items

🐞 **My mistakes (9 items-vs-positions slips!):** even filter · 1929 · 1480 `output[value - 1]` · 2149 one counter for two lists · `range(list)` ×3 · `scores.max()` · `pop(blocks[0])` (Piling Up) ✍️ **Rewrite:** second-highest score (with duplicates) · LeetCode 1929, 2149 · HackerRank *Lists*, *Piling Up!*

## 6️⃣ Functions (Day 06)

📖 **Read:** notes.md: *Day 06* · lessons/day06-functions.md · approaches.md: *Day 06 functions* **I can...** - ☐ write `def name(params):` with a default value, and call it - ☐ explain `return` vs `print`, and why a function with no `return` gives `None` - ☐ explain `break` vs `return` (loop vs whole function) - ☐ spot a **side effect** (`scores.sort()` inside a function changes the caller's list) - ☐ say why a variable must never share a function's name (`grade = ...`) - ☐ write a GUARD CLAUSE for empty input

🫀 **My mistakes:** `print` instead of `return` (×3) · `return {x}` makes a set · dead code after `return` · `return` outside a function (Nested Lists, Piling Up) ✍️ **Rewrite:** `count_vowels`, `is_palindrome`, `average`, `second_highest`, `grade` as functions that **return**

## 7️⃣ Dictionaries ⭐ (Day 07)

📖 **Read:** notes.md: *Day 07* · lessons/day07-dictionaries.md · add-remove-loop-book.md §3 · review-weekend-1.md *Sunday* drills **I can...** - ☐ add, change, look up and delete: `d[k] = v`, `d[k]`, `d.get(k, 0)`, `del d[k]`, `d.pop(k)` - ☐ say what `key in d` checks (**keys only**) - ☐ loop over keys, values and both (`.items()`) - ☐ write the COUNTING line from memory: `counts[x] = counts.get(x, 0) + 1` - ☐ find the most common item with BEST SO FAR (updating **both** variables) - ☐ explain why `d["z"] += 1` crashes on a new key

🫀 **My mistakes:** `best_count` never updated (letter counter) · counted letters instead of words (Word Order) · `counts.items` without `()` ✍️ **Rewrite:** letter counter + most common letter · Two Sum with a `seen` dict · HackerRank *Word Order*

## 8️⃣ Parsing input and printing ⭐ (HackerRank's favourite)

📖 **Read:** data-structures-guide.md §5–§6 · add-remove-loop-book.md §5 · notes.md: *Reading*

*many numbers from one line* **I can…** - ☐ read one number, a line of numbers, and "a name then numbers" (`name, *rest = input().split()`) - ☐ explain `list(map(int, input().split()))` piece by piece, and why `map` needs `list( … )` - ☐ read T test cases with a loop - ☐ print on one line 3 ways: `end=" "`, `" ".join(map(str, nums))`, `print(*nums)` - ☐ use `sep=` and f-string formatting `f"{avg:.2f}"`

🐞 **My mistakes:** `map( … ).sort()` · `input.split` without `()` · used the loop's last `name` instead of `query_name` (Finding the percentage) ✍️
**Rewrite:** HackerRank *Find the Runner-Up Score!*, *Finding the percentage*, *Nested Lists*

## 9️⃣ Patterns, errors and problem solving

📖 **Read:** approaches.md: *Pattern library* + any 3 problem sections · lessons/problem-solving-method.md · mistakes-journal.md: *Focus areas* + *Daily checklist* · cheat sheet §10–§12 **I can…** - ☐ name and write each pattern from memory: ACCUMULATOR, FILTER, COUNTING, BEST SO FAR, TWO POINTERS, EARLY EXIT, GUARD CLAUSE, FLAG, TAKE FROM BOTH ENDS, FIND THE BOTTLENECK - ☐ look at a new problem and say which pattern fits - ☐ use the STEP method: solve by hand → steps in English → code one step at a time - ☐ decode errors: `SyntaxError` (missing `:`, `return` outside a function), `IndexError`, `KeyError`, `TypeError` (`str + int`,

`range(list)`, `word[ch]`), `AttributeError` (`map.sort`, `scores.max`), "Time Limit Exceeded" (infinite loop) - ☐ explain the LeetCode format (`class Solution`, **return**, inputs already typed) vs HackerRank (read `input()`, **print**) - ☐ use the contest routine: read twice → example by hand → STEP comments → code → **Run** → Submit

✍️ **Rewrite:** LeetCode 344 (two pointers) · 1480 (accumulator) · 1689 (bottleneck) · 2161 (filter)

---

## 🏁 After the quiz

- Every ❌ goes to quiz-log.md's retry list and gets a drill in add-remove-loop-book.md if it's about lists, dicts or strings.
- All topics ✅ → **Day 08: sets and tuples** 🎉
- Daily minimum during revision: **1 LeetCode** (streak) and **1 commit**. The rainy-day bank in progress.md helps on busy days.

# 📗 Python Reference: Types, Ranges & Built-ins

↑ contents

> Every tool I've used so far, with **real examples**: what it takes, what it gives back, ✅ how to use it and ❌ how NOT to. **How to read the examples:** the code is on the left. After the `#` is what Python gives back (or prints). ✅ = the right way, ❌ = a trap or a crash (with the exact error). Golden question before using any tool: **"What does it give back?"** (a value, a new copy, or `None`?) Every example here was run in Python 3.12 to check it.

## 🎯 My quiz traps (Revision quiz, 28 Oct): read these first

| Trap | Example | Where |
|---|---|---|
| An empty result is **not** a crash | `list(range(5, 0))` → `[]` · `"x".find("q")` → `-1` · `"banana"[10:]` → `''` | §7 crash or not |
| Name the error | wrong **type** → `TypeError` ( `"12" + 3` ) · right type, bad **content** → `ValueError` ( `int("12.5")` ) | §7 |
| What does it give back? | `sorted("cab")` → a **list** `['a', 'b', 'c']` · `print()` → `None` · `round(2.5)` → `2` | §2 |
| `//` rounds **down** (toward −∞) | `-7 // 2` → `-4`, not `-3` and never `-3.5` | §6 |
| Uppercase is *smaller* | `max("Zebra", "apple")` → `'apple'` (Z = 90, a = 97) | §2 max |

---

# 1. 🧱 Data types and their ranges

**TYPE**
`int`

**WHAT IT HOLDS**
whole numbers

**RANGE / SIZE**
**no limit** in Python (it grows as needed)

**EXAMPLE**
`42`, `-7`

**CHANGEABLE?**
no

**TYPE**

`float`

**WHAT IT HOLDS**

decimal numbers

**RANGE / SIZE**

about **±1.8 × $10^{308}$**, accurate to **~15 digits**

**EXAMPLE**

`3.5`, `7 / 2`

**CHANGEABLE?**

no

**TYPE**

`bool`

**WHAT IT HOLDS**

yes/no

**RANGE / SIZE**

only `True` or `False` (they act like `1` and `0`)

**EXAMPLE**

`True + True` → `2`

**CHANGEABLE?**

no

**TYPE**

`str`

**WHAT IT HOLDS**

text

**RANGE / SIZE**

any length; positions `0 … len-1` (or `-len … -1`)

**EXAMPLE**

`"Hi"`

**CHANGEABLE?**

**no** (build a new one)

**TYPE**

`NoneType`

**WHAT IT HOLDS**

"nothing"

**RANGE / SIZE**

only `None`

**EXAMPLE**

a function with no `return` gives `None`

**CHANGEABLE?**

no

**TYPE**

`list`

**WHAT IT HOLDS**

ordered items

**RANGE / SIZE**

any length, any mix of types

**EXAMPLE**

`[3, "a", 2.5]`

**CHANGEABLE?**

**yes**

**TYPE**

`dict`

**WHAT IT HOLDS**

key → value pairs

**RANGE / SIZE**

any size; keys must be unchangeable ( `str` , `int` , `tuple` )

**EXAMPLE**

`{"a": 1}`

**CHANGEABLE?**

**yes**

**TYPE**
`tuple` *(Day 08)*

**WHAT IT HOLDS**
ordered items, frozen

**RANGE / SIZE**
like a list you can't change

**EXAMPLE**
`(1, 2)`

**CHANGEABLE?**
no

**TYPE**
`set` *(Day 08)*

**WHAT IT HOLDS**
unique items, no order

**RANGE / SIZE**
duplicates vanish

**EXAMPLE**
`{1, 2}`

**CHANGEABLE?**
yes

## `int` has no limit (C++ does)

```
2 ** 100        # 1267650600228229401496703205376   Python just grows the number
2 ** 31 - 1     # 2147483647   = the biggest int in C++ (about 2.1 billion)
```

In C++ (later!), going past 2,147,483,647 breaks the number, so LeetCode constraints matter more there.

### `float` is approximate

```
0.1 + 0.2                    #
0.30000000000000004   tiny error!
0.1 + 0.2 == 0.3             # ❌ False
round(0.1 + 0.2, 2) == 0.3   # ✅ True:
round first, then compare
float("1e309")               # inf   too
big for a float
float("inf") > 10 ** 100     # True
"bigger than any number" (Piling Up!)
```

**Why the error?** Computers store decimals in binary (0s and 1s). 0.1 can't be stored exactly in binary, just like 1/3 can't be written exactly in decimal (0.3333...). Rounding is the **fix**, the binary storage is the **reason**.

### Truthy and falsy (what `if x:` thinks)

```
bool(0), bool(0.0), bool(""), bool([]),
bool({}), bool(None)   # all False: "empty
/ zero / nothing"
bool(5), bool(-1), bool([0])       # True
any non-zero number, any non-empty list
bool(" ")                          # True
← a space IS a character, so not empty
bool("0")                          # True
← non-empty text
bool("False")                      # True
← the WORD False is still non-empty text
if nums:                           # ✅
short for "if the list is not empty"
```

### Positions: what's allowed

```
s = "banana"      # length 6 → positions 0…5, or -6…-1
s[0], s[-1]       # 'b' 'a'
s[5]              # 'a'   the last one = len - 1
s[6]              # ❌ IndexError: string index out of range
s[2:100]          # ✅ 'nana'   slices NEVER crash, they just stop at the end
s[10:]            # ✅ ''       empty, no crash
```

---

## 🔁 Converting between types

**`int(x)` : make a whole number**

**Takes:** text of a *whole* number, a float, or a bool · **Gives back:** `int`

```
int("42")             # ✅ 42
int(" -7 ")           # ✅ -7     spaces and a sign are OK
int("8 9")            # ❌ ValueError   a space in the MIDDLE is not OK
int("")               # ❌ ValueError   empty text
int(12.9)             # ✅ 12     cuts off the decimals (NO rounding!)
int(-2.5)             # ✅ -2     cuts toward 0
int(True)             # 1
int("12.5")           # ❌ ValueError: invalid literal for int() with base 10: '12.5'
int(float("12.5"))    # ✅ 12     text → float → int
int("abc")            # ❌ ValueError
```

**The quotes are NOT the problem.** `int()` is *made* to read text. It first trims spaces off both ends, and then the text must be only digits (with an optional `-` or `+` in front). A dot, a letter, a middle space or empty text → `ValueError`.

**`float(x)` : make a decimal number**

```
float("3.5")     # ✅ 3.5
float(7)         # ✅ 7.0
float("inf")     # ✅ inf
float("abc")     # ❌ ValueError: could not
convert string to float: 'abc'
```

**`str(x)` : make text (never crashes)**

```
str(5)               # ✅ '5'
"Age: " + str(30)    # ✅ 'Age: 30'
"Age: " + 30         # ❌ TypeError: can
only concatenate str (not "int") to str
```

**Fixing `"12" + 3` two ways (quiz Q4)**

```
"12" + 3          # ❌ TypeError: text +
number can't mix
int("12") + 3     # ✅ 15      make BOTH
numbers → it adds
"12" + str(3)     # ✅ '123'   make BOTH
text → it joins
```

Don't fix it by retyping the values ( `12 + 3` ). In real code the `"12"` comes from `input()` or `.split()`, so you have to **convert** it.

**`list(x)` : make a list from anything loopable**

```
list("abc")                  # ✅ ['a',
'b', 'c']
list(range(3))               # ✅ [0, 1, 2]
list(map(int, ["1", "2"]))   # ✅ [1, 2]
list({"a": 1, "b": 2})       # ['a', 'b']
keys only
list(5)                      # ❌
TypeError: 'int' object is not iterable
```

**`type(x)` : what am I holding?**

```
type(3)        # <class 'int'>
type(3.0)      # <class 'float'>
type(7 / 7)    # <class 'float'>   / always
gives a float
type("3")      # <class 'str'>
```

---

## 2. 🧰 Built-in functions (they go AROUND: `len(x)`)

**`print( ... )` : show on screen**

**Takes:** any values · **Gives back:** `None` (it only *shows*)

```
print("a", "b", 3)                    # a b 3
numbers need no quotes; each comma → one
space
print("a", "b", sep="-")              # a-b
sep = what goes between
print("a", end=" "); print("b")       # a b
end = what goes after (default: new line)
age = 30
print(f"Age is {age}!")               # ✅
Age is 30!
print(f"Age is{age}!")               # ❌
Age is30!   spaces are exactly what you
type
x = print("hi")                       # the
screen shows: hi
print(x)                              # the
screen shows: None   ← print gave back
nothing
```

**`input()`: read one line**

**Gives back: always text**, even if you type a number

```
n = input()                                  #
you type 5
n                                            #
'5'    TEXT
n * 2                                        #
❌ '55'  text repeated, not maths
int(n) * 2                                   #
✅ 10
nums = list(map(int, input().split()))       #
✅ you type "3 4 5" → [3, 4, 5]
```

### `len(x)` : how many?

```
len("hello")     # 5
len([1, 2, 3])   # 3
len({"a": 1})    # 1   number of keys
len("")          # 0
len(5)           # ❌ TypeError: object of type 'int' has no len()
```

### `range(start, stop, step)` : a run of whole numbers

**Takes: ints only** · **Gives back:** numbers from `start` up to, but **not including**, `stop`

```
list(range(4))            # [0, 1, 2, 3]   stop is EXCLUDED
list(range(2, 5))         # [2, 3, 4]
list(range(0, 10, 3))     # [0, 3, 6, 9]
list(range(10, 0, -3))    # [10, 7, 4, 1]   negative step = count down
list(range(4, -1, -1))    # [4, 3, 2, 1, 0]   backwards over positions
list(range(5, 0))         # []   ← can't reach 0 by going UP: empty, NOT a crash
for i in range(len(nums)):    # ✅ the positions of a list
for i in range(nums):         # ❌ TypeError: 'list' object cannot be interpreted as an integer
range(2.5)                # ❌ TypeError: 'float' object cannot be interpreted as an integer
range(1, 5, 0)            # ❌ ValueError: range() arg 3 must not be zero
```

### `sum(x)` : add everything up

```
sum([3, 4, 5])      # 12
sum(range(4))       # 6   (0 + 1 + 2 + 3)
sum([])             # 0   ← empty is fine
sum(["a", "b"])     # ❌ TypeError: numbers only
```

### `max()` / `min()` : biggest / smallest

```
max([3, 9, 2])          # 9
min(4, 7)               # 4      also works with 2+ separate values
best = max(best, h)     # ✅ BEST SO FAR pattern
max("Zebra", "apple")   # 'apple'   ← lowercase counts as BIGGER
max([])                 # ❌ ValueError: max() iterable argument is empty
max([], default=0)      # ✅ 0   safe for an empty list
```

**Why `'apple'`?** Every character has a number (`ord("Z")` → `90`, `ord("a")` → `97`). Text is compared one character at a time, and **all uppercase letters come before all lowercase ones**. To ignore case, compare `.lower()` versions.

### `sorted(x)` : a NEW sorted list

```
sorted([3, 1, 2])                  # [1, 2, 3]
sorted([3, 1, 2], reverse=True)    # [3, 2, 1]
sorted("cab")                      # ['a', 'b', 'c']   ← always a LIST, even from text
"".join(sorted("cab"))             # ✅ 'abc' glue it back into text
nums = sorted(nums)                # ✅ keep the result
sorted(nums)                       # ❌ result thrown away, nums unchanged
sorted([1, "a"])                   # ❌ TypeError: '<' not supported between instances of 'str' and 'int'
```

### `abs(x)` : distance from 0

```
abs(-3)      # 3
abs(2.5)     # 2.5
abs("3")     # ❌ TypeError: bad operand type for abs(): 'str'
```

### `round(x, digits)`

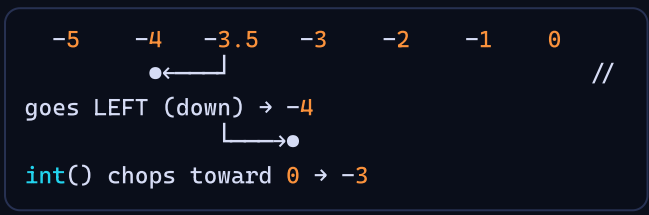
```
round(3.7)          # 4
round(3.14159, 2)   # 3.14
round(2.5)          # 2   ← a tie (.5) goes to the EVEN number
round(3.5)          # 4   (4 is even)
round(-2.5)         # -2
round(4.5)          # 4   between 4 and 5: 4 is even
```

**Ties go to even** only for an exact `.5` : Python looks at the **two neighbours** and picks the even one (it isn't "even goes low, odd goes up"). **Why?** If every `.5` went up, adding thousands of

rounded numbers would drift too high. Going to even sends half the ties up and half down, so they cancel out. Banks and statistics software use the same rule. 🔍 *Bookmarked to look into later.*

**`//` vs `int()`: two different ways to lose the decimals**

```
int(-3.5)    # -3   CHOPS the decimal → moves toward 0
-7 // 2      # -4   rounds DOWN → moves LEFT on the number line
7 // 2       # 3    for positive numbers both look the same
```




**`map(f, items)`: apply `f` to every item**

**Gives back:** a lazy **map object**, not a list!

```
map(int, ["1", "2"])            # ❌ <map object at 0x…>   can't see inside
list(map(int, ["1", "2"]))      # ✅ [1, 2]   text → numbers
" ".join(map(str, [1, 2]))      # ✅ '1 2'   numbers → text (for join)
m = map(int, ["1"]); m[0]       # ❌ TypeError: 'map' object is not subscriptable
```

`enumerate(x)` : position AND item together

```
for i, ch in enumerate("hi"):
    print(i, ch)                # 0 h
then   1 i
list(enumerate("hi"))           # [(0,
'h'), (1, 'i')]
```

`reversed(x)` : back to front (doesn't change `x` )

```
list(reversed([1, 2, 3]))   # ✅ [3, 2, 1]
"".join(reversed("abc"))    # ✅ 'cba'
"abc"[::-1]                 # ✅ 'cba'
the slice way
reversed([1, 2, 3])         # ❌
<list_reverseiterator …>   wrap it in
list()
```

---

## 3. 🔤 String methods (after the dot: `s.upper()` )

> Strings can't change, so **every string method gives back NEW text**. Save it, or it's lost:

```
s = "Hi"
s.upper()        # 'HI'    ❌ but s is still
'Hi' (result thrown away)
s = s.upper()    # ✅ now s is 'HI'
```

### `.upper()` / `.lower()` / `.strip()`

```
"hello".upper()        # 'HELLO'
"R".lower() == "r"     # ✅ True   lower both sides before comparing
"  hi  ".strip()       # 'hi'     spaces removed from BOTH ends (not the middle)
"--hi--".strip("-")    # 'hi'     strip other characters too
```

### `.split()`: cut text into a list of pieces

Think of each separator as a **scissor cut**. Whatever sits between two cuts becomes one piece:

```
"a,,b"  →  a | (nothing) | b   →  ['a', '', 'b']    2 commas = 2 cuts = 3 pieces
```

```
"3 4 5".split()            # ✅ ['3', '4', '5']   no argument = cut at ANY spaces
"  a  b ".split()          # ✅ ['a', 'b']  extra spaces ignored
"10,20,30".split(",")      # ✅ ['10', '20', '30']
"a,,b".split(",")          # ['a', '', 'b']        nothing between 2 commas = empty ''
"a, b".split(",")          # ❌ ['a', ' b']        the space sticks to 'b' → use .split(", ")
"a  b".split(" ")          # ❌ ['a', '', 'b']     2 spaces = 2 cuts → use .split()
"10,20".split(",")[0] + 5  # ❌ TypeError: pieces are TEXT → int( ... ) first
"x".split("")              # ❌ ValueError: empty separator
```

**Rule of thumb:** for input like `3 4 5`, always use `.split()` with **nothing inside**.

### `sep.join(items)` : glue a list of TEXT into one string

```
" ".join(["a", "b"])          # 'a b'
"".join(["a", "b"])           # 'ab'
", ".join(["x", "y"])         # 'x, y'
" ".join([1, 2])              # ❌
TypeError: sequence item 0: expected str
instance, int found
" ".join(map(str, [1, 2]))    # ✅ '1 2'
["a", "b"].join(" ")          # ❌
AttributeError: 'list' object has no
attribute 'join'  (glue goes FIRST)
```

### `.replace(old, new)`

```
"banana".replace("a", "o")      # 'bonono'
ALL matches
"banana".replace("a", "o", 1)   # 'bonana'
only the first 1
"banana".replace("x", "o")      # 'banana'
nothing found: no crash, no change
```

### `.count(sub)` : how many times? (no overlaps)

```
"banana".count("a")     # 3
"banana".count("ana")   # 1   ← after "ana"
at 1–3, it carries on from 4 ("na")
"banana".count("x")     # 0
```

**`.find(sub, start, end)` : WHERE is it? ( `-1` = not there)**

```
"banana".find("na")          # 2    first match
"banana".find("na", 3)       # 4    start looking at position 3
"banana".find("na", 0, 3)    # -1   only looks inside "ban"
"banana".find("x")           # -1   not found: NO crash
```

**`.index(sub)` : like `find` , but crashes**

```
"banana".index("n")    # 2
"banana".index("x")    # ❌ ValueError: substring not found   → use .find() or `in`
```

**`in` , `.startswith()` , `.endswith()` : yes / no**

```
"ana" in "banana"              # True
"Ana" in "banana"              # False   case matters
"hello".startswith("he")       # True
"hello".endswith("lo")         # True
```

**`.isdigit()` / `.isalpha()`**

```
"123".isdigit()    # True
"12.5".isdigit()   # False   the dot isn't a digit
"-5".isdigit()     # False   the minus isn't a digit
"abc".isalpha()    # True
"ab c".isalpha()   # False   the space isn't a letter
```

`s[i]` and slices `s[start:stop:step]`

```
s = "python"
s[0], s[-1]       # 'p' 'n'
s[1:4]            # 'yth'      stop
excluded
s[::2]            # 'pto'      every 2nd
s[::-1]           # 'nohtyp'   reversed
s[10]             # ❌ IndexError: string
index out of range
s[0] = "P"        # ❌ TypeError: 'str'
object does not support item assignment
s = "P" + s[1:]   # ✅ 'Python'   build a
new one instead
```

---

## 4. 📋 List methods

> List methods **change the list itself** and mostly give back `None`:

```
x = [3, 1]
x = x.append(5)   # ❌ now x is None!
(append gave back None, and you saved it)
x.append(5)       # ✅ x is [3, 1, 5]
```

**Adding: `.append()`, `.extend()`, `.insert()`**

```
x = [1, 2]
x.append(3)           # x = [1, 2, 3]
x.append([4, 5])      # x = [1, 2, 3, [4,
5]]   ← ONE item (a list inside a list)

x = [1, 2]
x.extend([3, 4])      # x = [1, 2, 3, 4]
each item added
x.extend("ab")        # x = [1, 2, 3, 4,
'a', 'b']   a string = its letters

x = ["a", "c"]
x.insert(1, "b")      # x = ['a', 'b', 'c']
POSITION first, then the value
x.insert(0, "z")      # x = ['z', 'a', 'b',
'c']   at the front
x.insert(99, "end")   # added at the end (a
too-big position doesn't crash)
x.insert("b")         # ❌ TypeError:
insert expected 2 arguments, got 1
```

**Removing: `.pop()` (by POSITION) vs `.remove()` (by VALUE) vs `del`**

```
x = [10, 20, 30]
x.pop()                # 30   gives back the LAST → x = [10, 20]
x.pop(0)               # 10   position 0 = the first → x = [20]
picked = x.pop()       # ✅ save what it gives back → picked = 20, x = []
x.pop()                # ❌ IndexError: pop from empty list
x = [5, 9]
x.pop(9)               # ❌ IndexError: pop index out of range   (pop wants a POSITION, not the value 9)

x = [3, 5, 3]
x.remove(3)            # x = [5, 3]   removes the FIRST 3 (a value); gives back None
x.remove(99)           # ❌ ValueError: list.remove(x): x not in list
if 99 in x:            # ✅ check first
    x.remove(99)

x = [10, 20, 30]
del x[1]               # x = [10, 30]   by position, gives back nothing
```

**Looking: `.index()`, `.count()`, `in`**

```
[5, 6, 5].index(5)   # 0      the first POSITION of that value
[5, 6, 5].count(5)   # 2
3 in [1, 2, 3]       # True   checks VALUES
[5, 6].index(9)      # ❌ ValueError: 9 is not in list
```

**Ordering: `.sort()` / `.reverse()` (in place, give back `None`)**

```
x = [3, 1, 2]
x.sort()                # x = [1, 2, 3]
x.sort(reverse=True)    # x = [3, 2, 1]
x.reverse()             # x = [1, 2, 3]
flips it
y = x.sort()            # ❌ y is None
y = sorted(x)           # ✅ a NEW sorted
list (the function, not the method)
```

**Replacing by position: `x[i] = v`**

```
x = [0] * 3       # [0, 0, 0]   3 slots
x[1] = 7          # [0, 7, 0]   replaces
slot 1 only
ans = []
ans[0] = 5        # ❌ IndexError: list
assignment index out of range   (can't grow
a list this way)
ans.append(5)     # ✅ [5]
```

**Joining and repeating: `+` and `*` (give back a NEW list)**

```
[1, 2] + [3]      # [1, 2, 3]
[0] * 3           # [0, 0, 0]
[1] + [2]         # ✅ [1, 2]
[1] + 2           # ❌ TypeError: can only
concatenate list (not "int") to list
```

---

## 5. 📖 Dictionaries

### Reading a value

```
d = {"a": 1}
d["a"]              # 1
d["z"]              # ❌ KeyError: 'z'
d.get("z")          # None   safe: no crash
d.get("z", 0)       # 0      safe, with your
own fallback
```

### Adding and changing

```
d = {"a": 1}
d["b"] = 2                      # adds:
{'a': 1, 'b': 2}
d["a"] = 5                      # overwrites:
{'a': 5, 'b': 2}
d["c"] += 1                     # ❌
KeyError: 'c'   there's no old value to add
to
d["c"] = d.get("c", 0) + 1      # ✅ the
COUNTING pattern → {'a': 5, 'b': 2, 'c': 1}
d[[1, 2]] = 3                   # ❌
TypeError: unhashable type: 'list'   (a
list can't be a key)
```

### Removing

```
d = {"a": 5, "b": 2}
d.pop("a")          # 5   gives back the
removed VALUE → d = {'b': 2}
del d["b"]          # d = {}
d.pop("q")          # ❌ KeyError: 'q'
```

### Checking and looping

```
d = {"a": 1, "b": 2}
"a" in d                    # True    checks KEYS only
1 in d                      # False   ← 1 is a value, not a key
len(d)                      # 2       number of keys
for k in d:                 # keys:   'a', 'b'
for v in d.values():        # values: 1, 2
for k, v in d.items():      # pairs:  ('a', 1), ('b', 2)
```

## 6. ➗ Operators: what they do depends on the type

### Numbers

```
7 + 2, 7 - 2, 7 * 2   # 9 5 14
7 / 2                 # 3.5
7 / 7                 # 1.0    / ALWAYS gives a float
7 // 2                # 3      divide, then round DOWN
-7 // 2               # -4     ← DOWN means toward −∞ (see the number line)
7 % 2                 # 1      the remainder
-7 % 2                # 1
n % 2 == 0            # ✅ the "is it even?" test
2 ** 3                # 8      power
2 ** 0.5              # 1.4142135623730951 square root
n = 1234
n % 10                # 4      ✅ the LAST digit (remainder after dividing by 10)
n // 10               # 123    ✅ DROP the last digit
n.pop()               # ❌ AttributeError: 'int' object has no attribute 'pop' (.pop() is for lists)
```

```
   -4 ←——— -3.5 ———→ -3
  ✅ //       (/)        ❌ not this
 "down" on the number line is always to the LEFT
```

### Text and lists

```
"ab" + "c"      # 'abc'      join (both sides the same type)
"ab" * 3        # 'ababab'   repeat with an int
[0] * 3         # [0, 0, 0]
"ab" + 3        # ❌ TypeError: can only concatenate str (not "int") to str
[1] + 2         # ❌ TypeError: can only concatenate list (not "int") to list
"ab" * "2"      # ❌ TypeError: can't multiply sequence by non-int of type 'str'
"ab" - "b"      # ❌ TypeError: unsupported operand type(s) for -: 'str' and 'str'
```

### Comparing ( `==` `≠` `<` `>` `≤` `≥` give back `True` / `False` )

```
3 == 3.0            # True
"3" == 3            # False   text is never equal to a number (no crash)
"apple" < "banana"  # True    character by character
"Zebra" < "apple"   # True    uppercase first (Z = 90 < a = 97)
"3" < 3             # ❌ TypeError: '<' not supported between instances of 'str' and 'int'
x = 5               # =  STORES
x == 5              # == COMPARES → True
```

**Combining:** `and` , `or` , `not`

```
age = 20
age ≥ 18 and age < 60   # True    both must be True
18 ≤ age < 60           # ✅ True  same thing, shorter
age < 5 or age > 60     # False   at least one must be True
not True                # False
```

---

## 7. 🚨 Error decoder: what each crash means

**Which one is it?** Ask in this order: 1. Can Python even **read** the line? No → `SyntaxError` 2. Is it the wrong **kind** of thing (text where a number should be, a list where an int should be)? → `TypeError` 3. Right kind of thing, but bad **content** ( `"12.5"` is text, but not *whole-number* text)? → `ValueError` 4. A **position** that doesn't exist? → `IndexError` · a **key** that doesn't exist? → `KeyError`

| Error | Meaning | My examples |
| --- | --- | --- |
| `SyntaxError` | Python can't read the line | missing `:`, `elif elif`, `range(1:n)`, `return` outside a function |
| `TypeError` | wrong **type** for this operation | `"12" + 3`, `[1] + 2`, `range(list)`, `len(5)`, `",".join([1, 2])` |
| `ValueError` | right type, **bad value** | `int("12.5")`, `x.remove(99)`, `max([])` |
| `IndexError` | position doesn't exist | `ans[0] = 5` on `[]`, `[].pop()`, `a.pop(a[0])` |
| `KeyError` | key doesn't exist | `d["z"] += 1` on a new key |
| `AttributeError` | that type has no such method | `map( ... ).sort()`, `scores.max()`, `["a"].join(" ")` |
| `NameError` | name never created (or a typo) | using a variable before assigning it |
| `ZeroDivisionError` | divided by 0 | `average([])` |
| "Time Limit Exceeded" | the loop never ends (or | `while` loop that never moves (LeetCode 344) |

| Error | Meaning | My examples |
| --- | --- | --- |
| | is too slow) | |

## ✅ These look scary but do NOT crash

```
"banana"[2:100]           # 'nana'
slices stop at the end
"banana"[10:]             # ''
"abc"[5]                  # ❌ but THIS one
crashes: one position = IndexError
list(range(3, 3))         # []
start = stop → nothing (stop is excluded)
int(" 8 ")                # 8
spaces at the ends are fine
list(range(5, 0))         # []
can't reach the stop → empty
"banana".find("x")        # -1
"not found" is an answer
"banana".replace("x", "o")   # 'banana'
nothing to replace
d.get("missing")          # None
sum([])                   # 0
x.insert(99, "v")         # adds at the end
"3" == 3                  # False
print("a", "b", 3)        # a b 3
numbers need no quotes
```

# 📕 The Add · Remove · Loop Book

↑ contents

> My weak spot, attacked one chapter at a time: **putting things into, taking things out of, and walking through** lists, dictionaries and strings. **How to use it:** 3 drills from the 🏋️ sections at the start of every session (answers are at the bottom, so cover them up!). A new chapter gets added whenever a problem trips me up. Companion to data-structures-guide.md, which covers parsing input and printing.

---

## 0. The one question that fixes most bugs

Before touching a list, ask: **"Do I mean a POSITION or a VALUE?"**

```
cubes  =  [ 4,  3,  2,  1 ]
position:   0   1   2   3      ← where it
sits (cubes[0])
value:      4   3   2   1      ← what it is
```

| Wants a **position** | Wants a **value** |
|---|---|
| `lst[i]` | `x in lst` |
| `lst.pop(i)` | `lst.remove(x)` |
| `lst.insert(i, x)` (i = where) | `lst.append(x)` (x = what) |
| `del lst[i]` | `lst.count(x)` |
| `range(len(lst))` gives positions | `lst.index(x)` gives back a position |

🔁 **My repeat offenders:** `pop(blocks[0])` (Piling Up), `output[value - 1]` (1480), `range(list)` (1672, Piling Up).

---

## 1. 📋 Lists

### Add

| Code | What happens | Gives back |
|---|---|---|
| `lst.append(x)` | puts x at the **end** | `None` |
| `lst.insert(i, x)` | puts x **at position i**, pushes the rest right | `None` |
| `lst + [x, y]` | makes a **new** list, glued together | the new list |
| `lst.extend([x, y])` | adds each item to the end | `None` |

## Remove

| Code | What happens | Gives back |
|---|---|---|
| `lst.pop()` | removes the **last** item | **that item** ✅ |
| `lst.pop(0)` | removes the **first** item | **that item** ✅ |
| `lst.pop(i)` | removes the item **at position i** | **that item** ✅ |
| `lst.remove(x)` | removes the **first item equal to x** | `None` |
| `del lst[i]` | removes the item at position i | nothing (it's a statement) |

## Look / change

| Code | Meaning |
|---|---|
| `lst[0]`, `lst[-1]` | first, last |
| `lst[len(lst) - 1]` | last, the long way (same as `lst[-1]`) |
| `lst[i] = x` | replace what's at position i |
| `lst[a:b]` | a **copy** of positions a … b-1 |

## ⚠️ Traps

1. **`lst = lst.append(x)`** → lst becomes **`None`**! `append` changes the list *in place* and gives back nothing. Just write `lst.append(x)`.
2. **`print(lst.sort())`** prints `None` for the same reason. Use `print(sorted(lst))`, or `lst.sort()` and then `print(lst)`.
3. **Removing while looping over the same list** skips items. Loop with `while lst:`

instead, or build a new list (FILTER pattern).

4. **`pop()` on an empty list** → `IndexError: pop from empty list`. Check `if lst:` first.

### 🏋️ Drills: lists (predict the output, then run it)

```
# L1
a = [10, 20, 30]
a.append(40)
print(a)

# L2
a = [10, 20, 30]
x = a.pop(0)
print(x, a)

# L3
a = [10, 20, 30]
a.insert(1, 99)
print(a)

# L4
a = [5, 3, 5, 1]
a.remove(5)
print(a)

# L5
a = [4, 3, 2]
a = a.append(1)
print(a)

# L6
a = [4, 3, 2, 1]
print(a.pop(a[0]))          # careful!
```

---

## 2. 🔁 Walking through a list: 4 ways

| I need… | Write | Each round gives me |
|---|---|---|
| just the items | `for x in lst:` | the value |
| positions (to look left/right, or change `lst[i]`) | `for i in range(len(lst)):` | the position |
| both | `for i, x in enumerate(lst):` | position **and** value |
| to eat it from the ends | `while lst:` + `lst.pop()` / `lst.pop(0)` | whatever you pop |
| two ends moving inwards | `left, right = 0, len(lst) - 1` + `while left ≤ right:` | two positions |

### 🏋️ Drills: loops

```python
# W1: what's printed?
for i in range(len(["a", "b", "c"])):
    print(i)

# W2: what's printed?
for i, x in enumerate(["a", "b"]):
    print(i, x)

# W3: what's printed?
s = [1, 2, 3]
while s:
    print(s.pop())

# W4: fix the bug
nums = [3, 1, 2]
for i in range(nums):
    print(nums[i])
```

---

## 3. 📖 Dictionaries

A dictionary maps a **key** → **value**. There are no positions, **only keys**.

## Add / change / remove / look

| Code | What happens |
|---|---|
| `d[key] = value` | adds the key, or **overwrites** it if it exists |
| `d[key] += 1` | ⚠️ crashes with `KeyError` if the key isn't there yet |
| `d[key] = d.get(key, 0) + 1` | the **COUNTING** line: safe even for a new key |
| `d[key]` | look up (⚠️ `KeyError` if missing) |
| `d.get(key, default)` | look up, with no crash |
| `key in d` | checks **keys**, not values |
| `d.pop(key)` | removes the key and **gives back its value** |
| `del d[key]` | removes the key |

## Walking through

| I need... | Write |
|---|---|
| keys | `for k in d:` |
| values | `for v in d.values():` |
| both | `for k, v in d.items():` |
| keys in order | `for k in sorted(d):` |

### 🏋️ Drills: dictionaries

```
# D1
d = {}
d["a"] = 1
d["a"] = 5
print(d)

# D2
d = {"x": 1}
print("x" in d, 1 in d)

# D3
counts = {}
for ch in "banana":
    counts[ch] = counts.get(ch, 0) + 1
print(counts)

# D4
d = {"a": 1, "b": 2}
v = d.pop("a")
print(v, d)

# D5: what happens?
d = {}
d["z"] += 1
```

---

## 4. 🔤 Strings: you can't change them, only rebuild

Strings are **frozen** (*immutable*): `s[0] = "X"` crashes. To "add" or "remove", you build a **new** string.

| Want | Write |
| --- | --- |
| add to the end | `s = s + "x"` or `s += "x"` |
| remove the first char | `s = s[1:]` |
| remove the last char | `s = s[:-1]` |
| list of words | `s.split()` |
| words → one string | `" ".join(words)` |
| list of characters | `list(s)` |

### 🏋️ Drills: strings

```
# S1
s = "cat"
s += "s"
print(s)

# S2
print("hello"[1:-1])

# S3
print("-".join(["a", "b", "c"]))
```

---

## 5. 🗺️ `map`: do the same thing to every item

`map(f, items)` = "apply **f** to each item". It gives back a lazy *map object*, so wrap it in `list( … )` to see it.

```
words = "4 3 2".split()          # ['4',
'3', '2']   ← strings!
nums  = list(map(int, words))    # [4, 3,
2]          ← numbers
```

It's the same as this loop:

```
nums = []
for w in words:
    nums.append(int(w))
```

### 🏋️ Drills: map

```
# M1
print(list(map(int, "7 8 9".split())))

# M2
print(list(map(len, ["hi", "hello"])))

# M3: what's printed (and why is it odd)?
print(map(int, ["1", "2"]))
```

---

## 6. 📒 Chapters added from real problems

| Problem | What tripped me | Chapter |
|---|---|---|
| HackerRank Piling Up! (26 Oct) | `pop(value)` instead of `pop(position)`; `range(list)` | §0, §1 |
| LeetCode 1672 (24 Oct) | `range(list)` | §2 |
| HackerRank Merge the Tools (23 Oct) | rebuilding a string per piece | §4 |
| LeetCode 2149 (22 Oct) | mixing positions of two lists of different sizes | §0 |
| HackerRank Word Order (22 Oct) | counting letters instead of words | §3 |

---

## ✅ Answers (cover these up!)

---

**Click to reveal**

- **L1** `[10, 20, 30, 40]`
- **L2** `10 [20, 30]`
- **L3** `[10, 99, 20, 30]`
- **L4** `[3, 5, 1]` : only the **first** 5 goes
- **L5** `None` : `append` gives back nothing
- **L6** `IndexError` : `a[0]` is 4, so it tries `pop(4)` , but the last position is 3
- **W1** `0 1 2` (one per line)
- **W2** `0 a` then `1 b`
- **W3** `3 2 1` : pop takes from the end
- **W4** `range(len(nums))`
- **D1** `{'a': 5}` : same key, overwritten
- **D2** `True False` : `in` checks keys only
- **D3** `{'b': 1, 'a': 3, 'n': 2}`
- **D4** `1 {'b': 2}`
- **D5** `KeyError: 'z'` : use `d.get("z", 0) + 1`
- **S1** `cats`
- **S2** `ell`
- **S3** `a-b-c`
- **M1** `[7, 8, 9]`
- **M2** `[2, 5]`
- **M3** `<map object at 0x ... >` : needs `list( ... )` around it

# 📦 Data Structures Guide: Lists, Strings, Dictionaries & Parsing Input

↑ contents

> ⭐ **My weak spot.** Read this before every HackerRank / LeetCode problem until it feels natural. Golden rule: **always know WHAT you're holding.** Is it a string? A list? A dictionary? A list *inside* a dictionary? 📕 **Adding, removing and looping** (append/pop/insert/remove, dict add/remove, `map`) now has its own drill book: add-remove-loop-book.md.

---

## 1. The 3 containers at a glance

| Holds |
| --- |
| **STRING `'...'`**<br>characters |
| **LIST `[...]`**<br>any items, in order |
| **DICTIONARY `{...}`**<br>**key** → **value** pairs |

Example

**STRING** `'...'`

`"Harsh"`

**LIST** `[...]`

`[25.0, 26.5, 28.0]`

**DICTIONARY** `{...}`

`{"Harsh": [25.0, 26.5, 28.0]}`

Get one item

**STRING** `'...'`

`s[0]` → `"H"`

**LIST** `[...]`

`x[0]` → `25.0`

**DICTIONARY** `{...}`

`d["Harsh"]` → `[25.0, 26.5, 28.0]`

By

**STRING** `'...'`

position

**LIST** `[...]`

position

**DICTIONARY** `{...}`

**key** (name)

Can change?

**STRING** `'...'`

❌ no

**LIST** `[...]`

✅ yes

**DICTIONARY** `{...}`

✅ yes

Length

**STRING '...'**
`len(s)`

**LIST [...]**
`len(x)`

**DICTIONARY {...}**
`len(d)` (number of pairs)

Check inside

**STRING '...'**
`"H" in s`

**LIST [...]**
`25.0 in x`

**DICTIONARY {...}**
`"Harsh" in d` (checks **keys**)

---

## 2. 🔍 "What am I holding?" Print it!

When confused, **print the variable and its type**. This is the #1 debugging trick:

```
print(student_marks)          # {'Harsh': [25.0, 26.5, 28.0], 'Anurag': [26.0, 28.0, 30.0]}
print(type(student_marks))    # <class 'dict'>
print(type(student_marks["Harsh"]))     # <class 'list'>
print(type(student_marks["Harsh"][0]))  # <class 'float'>
```

---

## 3. 🪜 Going down layers (nested data)

Read nested access **left to right, one step at a time**:

```
student_marks = {"Harsh": [25.0, 26.5, 28.0], "Anurag": [26.0, 28.0, 30.0]}

student_marks                    # the whole dictionary
student_marks["Harsh"]           # step 1: open the "Harsh" drawer → [25.0, 26.5, 28.0]
student_marks["Harsh"][1]        # step 2: second item in that list → 26.5
```

Like an address: **cupboard → drawer → item**.

---

## 4. 🔁 Looping over each container

| Loop | Gives you | Example |
|---|---|---|
| `for ch in "hi":` | each **character** | `h`, `i` |
| `for x in [10, 20]:` | each **item** | `10`, `20` |
| `for i in range(len(lst)):` | each **position** | `0`, `1`, then use `lst[i]` |
| `for key in d:` | each **key** | `"Harsh"`, `"Anurag"` |
| `for value in d.values():` | each **value** | `[25.0, ...]`, `[26.0, ...]` |
| `for key, value in d.items():` | **both** ⭐ | `"Harsh"`, `[25.0, ...]` |

🔑 **Name the loop variable after what it IS:** `ch`, `num`, `name`, `marks`. Use `i` only for positions.

⚠️ **The loop variable keeps its LAST value after the loop ends:**

```
for name in ["Harsh", "Anurag"]:
    pass
print(name)       # "Anurag", the last one!
(this caused my "Finding the percentage"
bug)
```

---

## 5. 📥 Parsing input (HackerRank's favourite)

Input always arrives as **text**. Turning it into numbers and containers is called **parsing**.

### One number

```
n = int(input())                          # "3"
→ 3
```

### Many numbers on one line

```
nums = list(map(int, input().split()))
```

Read it **inside** → **out** with input `"2 3 6"`: | Step | Code | Result | |---|---|---| | 1 | `input()` | `"2 3 6"` (one string) | | 2 | `.split()` | `["2", "3", "6"]` (list of strings) | | 3 | `map(int, ...)` | applies `int()` to each (a lazy "recipe") | | 4 | `list(...)` | `[2, 3, 6]` (real list of numbers) ✅ |

Use `float` instead of `int` for decimals: `list(map(float, ...))` → `[25.0, 26.5]`. Same

thing without `map` (a list comprehension, coming soon): `[int(x) for x in input().split()]`

### A name followed by numbers

```
name, *rest = "Harsh 25 26.5 28".split()
# name = "Harsh"
# rest = ["25", "26.5", "28"]     (* = "collect everything else into a list")
scores = list(map(float, rest))   # [25.0, 26.5, 28.0]
```

### Many lines → a dictionary

```
student_marks = {}
for _ in range(n):                  # _ = "I don't need the counter, just repeat n times"
    name, *line = input().split()
    student_marks[name] = list(map(float, line))
```

---

## 6. 🖨️ Printing

```
x = [1, 2, 3]
print(x)                    # [1, 2, 3]
(the whole list, with brackets)
print(*x)                   # 1 2 3
(* "unpacks" the items, separated by
spaces)
print(*x, sep=",")          # 1,2,3
for item in x:
    print(item)             # one per line

d = {"a": 1, "b": 2}
for key, value in d.items():
    print(f"{key}: {value}") # a: 1   b: 2

avg = 26.5
print(f"{avg:.2f}")         # 26.50 (2
decimal places)
```

## 7. 🧰 The patterns that cover 80% of problems

| Pattern | Container | Code shape |
|---|---|---|
| ACCUMULATOR | number | `total = 0` → `total += x` |
| BUILD A LIST | list | `result = []` → `result.append(x)` |
| FILTER | list | `if condition: result.append(x)` |
| COUNTING | dict | `counts[x] = counts.get(x, 0) + 1` |
| LOOKUP | dict | `d[key]` or `d.get(key, default)` |
| BEST SO FAR | any | `best = None` → `if best is None or x > best: best = x` |

---

## 8. ✅ Before submitting: data checklist

- ☐ Did I **print** the data once to see its real shape?
- ☐ Am I using the **right variable**? (e.g. `query_name`, not a leftover loop variable)
- ☐ Is it **text or a number**? Do I need `int()` / `float()`?
- ☐ Am I getting **items, positions or keys** from this loop?
- ☐ Am I using the data **after** it's been read? (order of lines matters!)

# 🧘 How to Solve a Problem Without Panicking

↑ contents

> "I can solve the logic, but I can't write the code" is **normal**. Logic and code are two different skills. This method connects them. **Never write the whole program at once.**

## The 6-step method

### 1. Solve the sample by hand 🖍️

Read the sample input and work out the answer on paper **without code**. If you can't do it by hand, you can't code it yet.

### 2. Write the steps as comments 📝

Turn your hand-solution into short English steps, as `#` comments in the editor:

```
# STEP 1: read all students into a list of [name, score]
# STEP 2: make a sorted list of just the scores
# STEP 3: find the second-lowest score
# STEP 4: collect the names with that score
# STEP 5: sort the names and print each one
```

Now the big scary problem is **5 small problems**.

### 3. For each step, ask "what am I holding?" 🔍

Write the **shape** of the data next to each step:

```
# students = [["Harry", 37.21], ["Tina",
37.2],  ... ]   (list of lists)
# scores   = [37.2, 37.21, 37.21, 39.0,
41.0]          (list of numbers)
# second   = 37.21
(one number)
# names    = ["Berry", "Harry"]
(list of strings)
```

### 4. Code ONE step, then `print()` and run ▶️

Write only Step 1, add `print(students)`, click **Run**. Does the output match the shape you wrote? ✅ Then move on. ❌ Fix just this step. **Small steps = small errors = no panic.**

## 5. Translate English → Python with the "phrasebook" 📖

| When I want to... | Python |
|---|---|
| start an empty list | `x = []` |
| add something to a list | `x.append(thing)` |
| go through every item | `for item in x:` |
| keep only some items | `if condition:` inside the loop, then `append` |
| sort a list | `x.sort()` or `sorted(x)` |
| get the first / last | `x[0]` / `x[-1]` |
| inside a pair `[name, score]` | `pair[0]` = name, `pair[1]` = score |
| stop at the first match | `break` |
| count things | `counts[k] = counts.get(k, 0) + 1` |
| print each item on its own line | `for item in x: print(item)` |

## 6. Clean up and submit 🧹

Remove the debug `print()`s, test the sample once more, and submit. If it fails, look at the failing input and ask: "what's different about it?"

## 🚑 When panic hits

- Stop typing. Breathe. **Go back to step 1** (by hand).

- Delete nothing. Comment the broken code out with `#` and start the current step again.
- A red squiggle = **one** small problem. Fix the **first** one only, then re-run.
- `return` only works inside `def`. On HackerRank's main code, use `print`.

## Common syntax traps from my journal

- Filling an empty list with `x[0] = ...` → use `x.append( ... )`
- `x.sorted()` ❌ → `sorted(x)` or `x.sort()`
- `for item in range(len(x))` gives **positions**, so use `x[item]` to get the value
- `return` outside a function ❌ → `print()`

# 🔁 Big Review #1: Sat 24 + Sun 25 Oct 2026

↑ contents

> ℹ️ Not done on the weekend (hackathon!). These drills are now **warm-ups for revision-1.md**, topics 4, 5, 7 and 8. Goal: make **strings, searching, printing, items vs positions, `map` and dictionaries** feel automatic. These are my own weak spots (said on 23 Oct). Everything here comes from **my** lessons, problems and mistakes.

**How each block works (about 1.5–2 h per day, take breaks):** 1. 📖 **Reread** (10 min): the listed notes. Read, don't copy. 2. 🧠 **Predict** (20 min): for each drill, write what it prints **before** running it. Then check in VS Code (`review.py`, a scratch file). 3. ✍️ **Rewrite from memory** (40 min): solve old problems again **without looking**. Stuck for 5 min? Peek at one line only. 4. 💡 **Better ways** (15 min): read the matching sections in approaches.md. 5. 🟢 **One Easy LeetCode** to keep the streak. Then commit.

Mark each drill ✅ (right first time), 🟡 (right after thinking) or ❌ (wrong). The ❌ ones go to quiz-log.md's retry list.

---

## 🗓️ Saturday: Strings, searching, printing, items vs positions

### 📖 Reread

- notes.md: Day 03 (loops), Day 04 (strings), "print tricks"
- data-structures-guide.md sections 2, 4 and 6
- mistakes-journal.md: focus area 1 (items vs positions)

## 🧠 Predict the output

```
s = "banana"
# S1
print(s[1], s[-1], s[1:4])
# S2
print(s.find("n"), s.find("z"))
# S3
print(s.count("a"), "nan" in s, "N" in s)
# S4
s.upper()
print(s)
# S5
print("a-b-c".split("-"))
# S6
print("-".join(["x", "y", "z"]))
# S7
for ch in "abc":
    print(ch, end=" ")
# S8
for i in range(len("abc")):
    print(i, end=" ")
# S9
nums = [5, 7, 9]
for i in range(len(nums)):
    print(i, nums[i])
# S10
print(1, 2, 3, sep=", ")
# S11
print("A", end="")
print("B")
# S12
result = ""
for ch in "hello":
    if ch not in result:
        result += ch
print(result)
```

**S13 (choose the loop):** you need to print the **position** of every `"a"` in `"banana"`. Items loop or positions loop? Why? Write it.

**S14 (find the bug):**

```
word = "level"
for ch in word:
    if word[ch] == "e":
        print("found")
```

### ✍️ Rewrite from memory

1. Day 04: **vowel counter** (count vowels in a word, any case)
2. Day 04: **palindrome check**
3. LeetCode 344: **Reverse String** in place (two pointers)
4. HackerRank: **Merge the Tools!** (try it with no peeking now)

### 💡 Better ways

approaches.md: Palindrome, Vowel counter, 344, Merge the Tools

---

## 🗓️ Sunday: `map`, parsing input, dictionaries (+ 🗓️ Sunday review)

### 📖 Reread

- notes.md: "Reading many numbers from one line", Day 07 (dictionaries)
- data-structures-guide.md sections 1, 3 and 5
- lessons/day07-dictionaries.md: the tally-sheet part

## 🧠 Predict the output

```
# M1  (pretend you typed: 4 5 6)
line = "4 5 6"
print(line.split())
# M2
print(map(int, line.split()))
# M3
print(list(map(int, line.split())))
# M4
print(list(map(str, [1, 2, 3])))
# M5
print(" ".join(map(str, [1, 2, 3])))
# M6
name, *rest = "Harsh 25 26".split()
print(name, rest)

d = {"a": 1, "b": 2}
# D1
print(d["a"], d.get("z"), d.get("z", 0))
# D2
for k in d:
    print(k, end=" ")
# D3
for k, v in d.items():
    print(k, v)
# D4
print(list(d.values()), len(d))
# D5
print("a" in d, 1 in d)
# D6
d["c"] = 3
d["a"] = 10
print(d)
# D7
counts = {}
for ch in "hello":
    counts[ch] = counts.get(ch, 0) + 1
print(counts)
```

**D8 (what happens?):** `print(d["z"])`

**D9 (find the bug):**

```
counts = {}
for word in ["hi", "yo", "hi"]:
    counts[word] += 1
```

**D10 (choose):** you have `nums = [3, 8, 3]` and must print `3 8 3` on one line. Write it **two** ways.

### ✍️ Rewrite from memory

1. Day 07 Part 3: **letter counter + most common letter**
2. HackerRank: **Word Order**
3. HackerRank: **Finding the percentage** (many lines → a dictionary)
4. LeetCode 2161: **Partition by Pivot** (the tidier `for num in nums` version)

### 💡 Better ways

approaches.md: Word Order, Two Sum (the `seen` dict version), 2149, 2161

### 🗓️ Then: Sunday review

Claude gives a 10-question quiz on the whole week, logged in quiz-log.md.

---

### ✅ Answers (only after you've predicted!)

**Saturday** - S1 `a a ana` · S2 `2 -1` · S3 `3 True False` - S4 `banana`: string methods return a NEW string, so the result must be saved ( `s = s.upper()` ) - S5 `['a', 'b', 'c']` · S6 `x-y-z` · S7

`a b c` · S8 `0 1 2` - S9 `0 5` / `1 7` / `2 9` · S10 `1, 2, 3` · S11 `AB` · S12 `helo` - S13 **positions loop**, because you need the position: `for i in range(len(s)):` / `if s[i] == "a": print(i)` → `1 3 5` - S14 `ch` is already a **letter**, so `word[ch]` crashes ("string indices must be integers"). Use `if ch == "e":`

**Sunday** - M1 `['4', '5', '6']` (still text) · M2 `<map object at 0x ... >` (a recipe, not a list) · M3 `[4, 5, 6]` - M4 `['1', '2', '3']` · M5 `1 2 3` · M6 `Harsh ['25', '26']` - D1 `1 None 0` · D2 `a b` (looping a dict gives **keys**) · D3 `a 1` / `b 2` · D4 `[1, 2] 2` - D5 `True False` (`in` checks **keys**, not values) · D6 `{'a': 10, 'b': 2, 'c': 3}` (changing a key keeps its place) - D7 `{'h': 1, 'e': 1, 'l': 2, 'o': 1}` · D8 **KeyError: 'z'** - D9 KeyError on the first `"hi"`: the key doesn't exist yet. Use `counts[word] = counts.get(word, 0) + 1` - D10 `print(" ".join(map(str, nums)))` or `for n in nums: print(n, end=" ")` (or `print(*nums)`)

↑ contents

# 📘 Day 06: Functions 🧩

A **function** is a named, reusable block of code. You've been *using* them all along (`print()`, `len()`, `max()`). Now you make your own.

## 1. Define and call

```
def greet():                    # def = define a function
    print("Hello, Gowri!")

greet()                         # call it (and reuse it as often as you like)
```

## 2. Parameters = input

```
def greet(name):
    print(f"Hello, {name}!")

greet("Optimus")                # Hello, Optimus!
```

## 3. `return` = output ⭐

```
def square(n):
    return n * n                # hands the answer BACK to the caller

result = square(5)              # 25
print(square(3) + square(4))    # 25
```

## ⚠️ `print` vs `return`: the #1 trap

```
def add_print(a, b):
    print(a + b)              # only shows
it on screen

def add_return(a, b):
    return a + b              # gives the
value back so you can use it

x = add_print(2, 3)           # shows 5,
but x = None 😱 (just like .sort()!)
y = add_return(2, 3)          # y = 5 ✅
```

## 4. `return` exits immediately

```
def is_even(n):
    if n % 2 == 0:
        return True           # stops here
    return False              # only
reached if the if was False

# shorter: return n % 2 == 0  (the
comparison already IS True or False)
```

## 5. Defaults

```
def power(base, exp=2):
    return base ** exp

power(3)      # 9  (exp defaults to 2)
power(2, 5)   # 32
```

## 6. Inside stays inside

```
def make_total():
    total = 100
make_total()
print(total)   # ❌ NameError: use return to get values out
```

---

## ✍️ Exercise: `day06.py`

Turn your old code into reusable tools 🧰 1. `is_even(n)` → True/False. Test with 4 and 7. 2. `is_palindrome(word)` → True/False, ignoring case (reuse Day 04) 3. `count_vowels(text)` → number of vowels (reuse Day 04) 4. `second_highest(scores)` → handles duplicates (reuse Day 05 / Runner-Up) 5. `average(numbers)` → the average of a list

Call each one and print the result:

```
print(is_palindrome("Racecar"))          # True
print(second_highest([95, 95, 88, 61]))  # 88
```

**Bonus:** `grade(score)` → `"A"` for 90 or more (90 counts!), `"B"` for 75–89, `"C"` for 50–74, below 50 `"F"`. Use early `return`s, with no `elif` needed!

---

## 🔥 HackerRank: Write a function

Return `True` if a year is a **leap year**: - divisible by **4** → leap... - **unless** divisible by **100** → not leap... -

**unless** also divisible by **400** → leap again!

Test values: 2000 ✅, 1900 ❌, 2024 ✅, 2023 ❌

**Hints:** - `year % 4 == 0` checks divisibility - Use early `return` and check the **most specific rule first** (400 → 100 → 4) - Keep HackerRank's `def is_leap(year):` and fill in the logic

# 📘 Day 07: Dictionaries 📖

↑ contents

A **dictionary** stores **pairs**: a **key** → a **value**. You look things up by **name** instead of by position, just like a real dictionary (word → meaning) or a phone book (name → number).

## 1. Creating and reading

```
robot = {
    "name": "Optimus",
    "maker": "Tesla",
    "year": 2022,
}
print(robot["name"])        # Optimus
(look up by KEY, not position)
print(len(robot))           # 3 pairs
```

- `{ }` with `key: value` pairs, separated by commas
- Keys are usually strings, and values can be anything (numbers, strings, lists...)

## 2. Missing keys: `[]` vs `.get()`

```
print(robot["price"])           # ❌
KeyError: 'price'
print(robot.get("price"))       # None, no
crash
print(robot.get("price", 0))    # 0, your
own default
print("price" in robot)         # False,
checks KEYS
```

## 3. Adding, changing, deleting

```
robot["price"] = 20000      # add a new pair
robot["year"] = 2024        # change an existing value (same syntax!)
del robot["maker"]          # delete a pair
```

Dictionaries **can be changed**, like lists.

## 4. Looping

```
for key in robot:                       # loops over KEYS
    print(key)

for value in robot.values():            # just the values
    print(value)

for key, value in robot.items():        # BOTH at once ⭐ the most useful
    print(f"{key}: {value}")
```

## 5. The COUNTING pattern ⭐ (you'll use this constantly)

**Think of a tally sheet 🖍️.** Read one letter at a time. If the letter isn't on the sheet, write it down with 1. If it's already there, add 1 to its number. The dictionary is the tally sheet: **keys = letters, values = counts**.

```
word = "banana"
counts = {}                          # start empty (ACCUMULATOR!)
for ch in word:
    if ch in counts:
        counts[ch] += 1              # seen before: add 1
    else:
        counts[ch] = 1               # first time: start at 1
print(counts)                        # {'b': 1, 'a': 3, 'n': 2}
```

**Watch the sheet change:** | Round | `ch` | On sheet? | What happens | `counts` after ||---|---|---|---|---|| 1 | b | no | `counts["b"] = 1` | `{'b': 1}` || 2 | a | no | `counts["a"] = 1` | `{'b': 1, 'a': 1}` || 3 | n | no | `counts["n"] = 1` | `{'b': 1, 'a': 1, 'n': 1}` || 4 | a | **yes** | `counts["a"] += 1` | `{'b': 1, 'a': 2, 'n': 1}` || 5 | n | **yes** | `counts["n"] += 1` | `{'b': 1, 'a': 2, 'n': 2}` || 6 | a | **yes** | `counts["a"] += 1` | `{'b': 1, 'a': 3, 'n': 2}` |

**Why the `if`?** `counts["a"] += 1` means "add 1 to the number next to `a`". If `a` isn't on the sheet yet, there's no number, so it crashes with `KeyError`. The first time, you must write it down with 1.

Shorter, using `.get()` with a default:

```
for ch in word:
    counts[ch] = counts.get(ch, 0) + 1
```

Read it **right side first**: 1. `counts.get(ch, 0)`: "what number is next to this letter? If it's not

there, pretend it's **0**" 2. `+ 1` : add one 3. `counts[ch] = ...` : write the new number back

Round 1 ( `b` , not there): 0 + 1 = **1**. Round 4 ( `a` , already 1): 1 + 1 = **2**. It's the same as the `if/else` version, with the `0` replacing the "not there yet" case.

**See it yourself:** add `print(ch, "→", counts)` inside the loop and watch the sheet grow.

## 6. Lists inside dictionaries

```
marks = {
    "Gowri": [85, 92, 78],
    "Optimus": [99, 100, 98],
}
print(marks["Gowri"][1])
# 92
print(sum(marks["Gowri"]) /
len(marks["Gowri"])) # average of Gowri's
marks
```

## Lists vs dictionaries

| | List `[ ]` | Dictionary `{ }` |
|---|---|---|
| Find items by | **position** `x[0]` | **key** `x["name"]` |
| Order matters? | yes | keeps insertion order, but you look things up by key |
| Best for | a sequence of similar things | **labelled** data, lookups, counting |

## ✍️ Exercise: `day07.py`

1. **Robot profile:** make a dictionary for your favourite robot (name, maker, year, height). Print each pair with `.items()`, add a `"price"`, change the `year`, and delete one key.
2. **Phone book:** a dictionary of 3 names → numbers. Ask the user for a name and print the number, or `"Not found"` if it's missing (use `.get()` or `in`, with **no crash**).
3. **Letter counter:** ask for a word and count each letter with the COUNTING pattern. Then print the **most common letter**. *(Hint: loop over `.items()` and keep the BEST SO FAR, like second-highest!)*
4. **Grade book:** `scores = {"Gowri": 92, "Atlas": 74, "ASIMO": 58, "Spot": 81}`. **Reuse your Day 06 `grade()` function** to print each name with their grade: `Gowri: A`.

**Bonus 🧠 (LeetCode preview!):** given `nums = [2, 7, 11, 15]` and `target = 9`, find the **two positions** whose numbers add up to the target (answer: `0, 1`, because 2 + 7 = 9). *Hint:* as you loop, store each number you've seen in a dictionary `{number: position}`. For each new number, check whether `target - number` is already in the dictionary. This is **LeetCode problem #1, "Two Sum"**, the most famous interview problem!

## 🔥 HackerRank: Finding the percentage

They give you a dictionary of `name → list of marks`. Print the **average** of one student's marks, with **2 decimal places**.

**Hints:** - Their starter code already builds the dictionary `student_marks` for you - Look up the student's list, then `sum( ... ) / len( ... )` (Day 05 + Day 06!) - 2 decimals: `print(f"{avg:.2f}")`. The `:.2f` inside the f-string means "2 digits after the point". That's new and useful!